

A Shift in the Algorithmic View of LWPM? Coding-Theoretic and Optimisation-Based Approaches

Simona-Maria Lazarescu^{1,2} , Ioana Boureanu¹[0000–0001–5864–777X], and
Constantin Catalin Dragan¹[0000–0002–4033–9880]

¹ Surrey Centre for Cyber Security, University of Surrey, Guildford, UK
{s.lazarescu,i.boureanu,c.dragan}@surrey.ac.uk

² Romanian Academy, Bucharest, Romania

Abstract. The Low-Weight Polynomial Multiple (LWPM) problem is a fundamental computational problem in stream-cipher cryptanalysis and underlies security analyses of numerous stream-cipher constructions, including Grain, Trivium, SNOW, and ZUC. These stream ciphers are deployed across a wide range of practical applications, ranging from IoT, RFID, and embedded devices to mobile communication systems. Its concrete hardness is typically estimated through reductions to coding-theoretic problems and Information Set Decoding (ISD) algorithms.

In this work, we present the first systematic experimental comparison of fundamentally different algorithmic paradigms for solving LWPM: coding-theoretic approaches based on ISD (the classical Stern algorithm and the representation-based BJMM algorithm), and exact optimisation approaches based on Mixed Integer Linear Programming (MILP) and Maximum Satisfiability (MAXSAT). We evaluate these methods across a broad range of cryptographically relevant parameter regimes, including feedback polynomials taken from published LFSR-based ciphers, and compare their solution quality, runtime, memory consumption, and robustness.

Our results show that, in the bounded-degree, low-weight regimes that arise naturally in practical cryptanalysis, off-the-shelf optimisation solvers are competitive with specialised ISD-based solving: they solve a larger fraction of instances within the imposed time limit than the ISD baselines, with lower and more predictable running times, and they return multiples of the same weight whenever ISD also succeeds. There is one exception, and that is real feedback polynomials, for which ISD can perform better than some MAXSAT-solvers, in some cases only: finding slightly better results, but often in slower times. These findings indicate that the practical hardness of LWPM depends on the chosen algorithmic model, and that security evaluations relying exclusively on ISD-based analyses should be complemented by optimisation-based ones in these parameter ranges.

Keywords: Low-Weight Polynomial Multiple · algorithmic cryptanalysis · optimisation problems

1 Introduction

The Low-Weight Polynomial Multiple (LWPM) problem consists in finding a polynomial $Q(x)$ such that the product $P(x)Q(x)$ has small Hamming weight, subject to degree constraints. This problem arises naturally in the cryptanalysis of stream ciphers, where low-weight polynomial multiples induce parity-check relations and statistical correlations that enable fast correlation and algebraic attacks [32,16,31,40]. Thus, the presumed hardness of LWPM underlies many concrete security arguments for symmetric cryptographic constructions.

The Importance of the LWPM problem in Cryptography. Initially, the LWPM problem arose in the context of linear feedback shift registers (LFSRs) [36], due to the relationship between sparse multiples and low-density parity-check relations, which are exploited in correlation or fast-correlation attacks, statistical distinguishers against stream ciphers, and communication-oriented cryptographic primitives.

Recently, the problem of sparse structure search has gained relevance due to the widespread adoption of lightweight and hardware-optimized cryptographic mechanisms in wireless communications, IoT, and embedded systems. Many contemporary lightweight stream-based constructions leverage pseudo-random sequence generators derived from LFSR, motivated by their reduced energy consumption, throughput efficiency, and low implementation cost.

As a result of its architecture that combines linear and nonlinear components, Grain-128A serves as a relevant example of a lightweight stream cipher. The standardization in [19] for radio frequency identification (RFID) air-interface security and the inclusion in the NIST lightweight crypto standardization process through Grain-128AEADv2 stream cipher [17] reinforce the importance of Grain-128A in a sparse construction context.

Lightweight hardware-oriented stream ciphers such as the Trivium cipher, designed for FPGA implementation, and MICKEY 2.0, designed for RFID and embedded systems, are also based on LFSR/NLFSR. These streams can be regarded as relevant examples because they were selected as finalists for the highly restricted hardware profile in the ECRYPT eSTREAM evaluation project [11].

Representative telecommunication stream ciphers such as SNOW [15] and ZUC [14] are standardized within the European Telecommunications Standards Institute and 3rd Generation Partnership Project, and are based on LFSR and hardware-efficient sequence generation mechanisms. The modern communication systems, including UMTS, LTE, 5G technologies, and emerging 6G infrastructures, reinforce the importance of lightweight cryptographic mechanisms.

The increasing relevance of sparse structure search in modern cryptanalysis is further underscored in recent code-based cryptosystems, such as BIKE and HQC, the algorithm selected for standardization in the fourth round of the NIST PQC process. Although the approach proposed in [35] is not explicitly formulated as an LWPM instance, it exhibits important similarities in terms of hidden sparse polynomial support reconstruction.

Approaches to Solving the LWPM Problem. Despite its importance, no deterministic polynomial-time algorithm is known for LWPM in any non-

trivial regime. All known approaches are exponential and span several distinct algorithmic families, including time–memory trade-offs (TMTOs) [16,9], lattice-based techniques [12], coding-theoretic reductions to low-weight codeword or syndrome decoding problems [27], and, more recently, formulations as combinatorial optimisation problems [39]. Among these, the dominant methodology for assessing the concrete hardness of LWPM in cryptographic settings is to reduce it to a structured instance of the Syndrome Decoding Problem (SDP) and to analyse it using Information Set Decoding (ISD).

ISD algorithms, starting with Prange’s original method [34] and followed by improvements due to Leon and Stern [24,38], as well as more recent variants [20,30], provide the best-known classical algorithms for solving SDP. Consequently, ISD complexity estimates are routinely used to justify security margins against attacks relying on low-weight polynomial multiples. This practice implicitly assumes that ISD accurately captures the difficulty of LWPM across all parameter regimes and structural classes relevant in cryptanalysis. Yet, recently, [39] has already shown that LWPM can be formulated as an optimisation problem.

Our Goal. The aim of this paper is to revisit the algorithmic landscape of LWPM from an explicitly experimental perspective. Rather than proposing new reductions or new decoding algorithms, we conduct a systematic comparison between optimisation-based solvers (MILP and MAXSAT) and coding-theoretic solvers obtained by reducing LWPM to ISD. Our focus is on concrete performance metrics: solution quality, runtime, memory consumption, and robustness across different classes of LWPM instances.

Our experiments give a consistent picture. On bounded-degree, low-weight instances, both random and structurally non-random, and on feedback polynomials taken from the literature, off-the-shelf MILP and MAXSAT solvers solve a larger fraction of instances within a fixed time budget than our ISD baselines (Stern and BJMM), with running times that are lower and far less dispersed. Whenever ISD does return a multiple, its weight matches the one found by the optimisation solvers. The question we set out to answer is therefore not “is ISD worse than MILP”, but rather can generic, unoptimised SAT/MILP solving compete with specialised LWPM solving in the regimes where security claims are made? The answer, in the regimes we tested, is yes. This has a direct consequence for security evaluation: ISD-based complexity estimates in these regimes should be complemented, not replaced, by optimisation-based ones. We stress that this is not a solve-all result: ISD remains the method of choice for large, random, high-weight instances, which we do not cover.

Contributions. The main contributions of this paper are:

- We present a comprehensive experimental comparison between optimisation-based LWPM solvers (MILP and MAXSAT) and coding-theoretic solvers based on classical ISD (i.e., the foundational Stern and the more modern, representation-based BJMM algorithms).
- We also identify concrete parameter regimes and structural classes of LWPM instances, including feedback polynomials from published LFSR-based de-

signs; in these cases, we see an interesting variation of wins in performance between certain optimisation solvers and certain ISD-based algorithms.

So, this work suggests that LWPM hardness could be assessed using a broader algorithmic perspective rather than relying solely on ISD-based analysis.

2 Preliminaries

2.1 Formal definition of the LWPM problem

Definition 1 (Low-Weight Polynomial Multiple (LWPM) [27]). *Given a polynomial $P(x) \in \mathbb{F}_2[x]$, an integer $n > \deg(P)$, and a weight bound w , the decision version of the Low-Weight Polynomial Multiple (LWPM) problem asks whether there exists a non-zero polynomial $Q(x)$ with $\deg(Q) < n$ such that $\text{wt}(P(x)Q(x)) \leq w$.*

The optimization version [39] asks to compute $\min_{0 \neq Q, \deg(Q) < n} \text{wt}(P(x)Q(x))$, or equivalently to find a polynomial Q attaining this minimum.

The LWPM problem is parameterised by the degree of $P(x)$, the degree bound n imposed on $Q(x)$, and the Hamming weight of the product $P(x)Q(x)$.

Cryptographic relevance. Low-weight polynomial multiples give rise to low-weight parity-check relations, which induce statistical correlations between keystream bits and internal state bits in stream ciphers. So, LWPM plays a central role in fast correlation and algebraic attacks, and its concrete hardness directly impacts the security evaluation of many symmetric primitives [32,16,31,40].

Parameters influencing hardness. The computational difficulty of LWPM depends on several structural parameters, including: 1. the degree and coefficient density of $P(x)$; 2. the degree bound n on $Q(x)$; 3. the target Hamming weight w of $P(x)Q(x)$; 4. algebraic properties of $P(x)$ such as sparsity, cyclic structure, or special factorisation patterns.

Different parameter regimes give rise to markedly different algorithmic behaviours and correspond to distinct cryptographic use cases.

Optimising only the degree n . In this regime, the objective is to find any non-zero multiple of $P(x)$ with degree less than a prescribed bound, without explicit constraints on the weight. This variant arises in finite field arithmetic and in the construction of polynomial or normal bases. From a cryptanalytic perspective, however, such multiples typically have large weight and therefore limited practical relevance.

Optimising only the weight w (fixed n). This is the most common and cryptographically relevant regime. One fixes a degree bound n (typically chosen as a polynomial function of $\deg(P)$) and aims to minimise $\text{wt}(P(x)Q(x))$ over all non-zero Q with $\deg(Q) < n$. Smaller weight yields stronger statistical bias and thus more efficient correlation-style attacks; so, most practical cryptanalytic treatments of LWPM are best viewed as solving this optimisation problem.

Simultaneously constraining (n, w) . The most challenging and practically relevant instances arise when both the degree and the weight are constrained. Such instances occur in full key-recovery and distinguishing attacks, where cipher

structure limits the admissible degree of parity-check relations, while successful cryptanalysis requires the weight to remain small. This regime, summarized in Table 1, is widely regarded as the hardest instantiation of LWPM, and is the focus of most concrete security analyses. These regimes and their roles in cryptography are summarised in Table 1.

Parameter regime	Problem variant	Cryptographic context
Fixed n , minimise w	Optimisation	Fast correlation attacks, bias maximisation
Fixed w , increase n	Mixed (decision + optimisation)	TMT0 and collision-based approaches
Bounded (n, w)	Optimisation	Full key-recovery, concrete security evaluation
Complexity analysis	Decision	Hardness proofs and reductions
Algorithm internals	Decision (subroutine)	Pruning and feasibility checks

Table 1: Decision + optimisation variants of LWPM & their cryptographic roles.

Decision versus optimisation formulations. Although the decision version of LWPM appears naturally in complexity-theoretic contexts and reductions, practical cryptanalysis overwhelmingly relies on optimisation variants. Attack efficiency depends continuously on the achieved weight rather than on mere feasibility, and security margins are typically derived from lower bounds on the minimal achievable weight as a function of the degree.

As a result, even when security claims are phrased in decision form (e.g., the non-existence of multiples below a certain weight), their justification is fundamentally optimisation-based. This distinction motivates the focus of this work on comparing algorithmic approaches that directly target optimisation.

2.2 Lattice-based approaches and the role of LLL in LWPM

LLL-based heuristics [23] have been proposed for LWPM by embedding the set of multiples into a lattice and searching for short vectors via basis reduction [12].

LWPM variant addressed by LLL. These methods target the optimisation flavour of LWPM under a fixed degree bound n , aiming to obtain a low-weight multiple $P(x)Q(x)$. They provide neither optimality guarantees nor non-existence proofs for weight thresholds, so they do not directly address the decision version.

Parameter regime & limitations in cryptographic regimes. The lattice dimension grows linearly with n , and LLL minimises Euclidean norm, which only loosely correlates with Hamming weight over $\mathbb{F}_2[x]$. As a result, LLL is effective only for relatively small n and becomes infeasible once n reaches a few hundred [12,27]. In the tightly constrained (n, w) regimes typical in cryptanalysis, controlling Hamming weight precisely is essential, and Euclidean shortness offers no guarantee of cryptanalytic usefulness. Moreover, LLL cannot support security claims of the form “no multiple of weight $\leq w$ exists below degree n ” since it yields no lower bounds and no infeasibility certificates.

Relation to other approaches. Because of these limitations, lattice heuristics are not used in state-of-the-art cryptanalysis, where coding-theoretic reductions and ISD-style algorithms scale better to realistic parameters [27].

2.3 SDP, ISD, and reductions from LWPM

A linear binary $[n, k]$ code C is a k -dimensional subspace of $\mathbb{F}_2^n$, which allows error detection and correction by mapping k -bit information messages into n -bit codewords. The algebraic structure of the code is defined by a generator matrix $G \in \mathbb{F}_2^{k \times n}$, which contributes to the encoding process for a message m ($c = m \cdot G$) and forms a basis for C , and a parity-check matrix $H \in \mathbb{F}_2^{(n-k) \times n}$, that validates the condition for the codeword c , $c \cdot H^T = 0$. The Hamming weight is defined as the number of non-zero elements of a vector $x \in \mathbb{F}_2^n$ and emphasizes the error-correcting capability of the code.

The Syndrome Decoding Problem (SDP) Conceptually, SDP shows the challenge of finding a random sparse error vector e when only the parity-check matrix and the syndrome are known.

Definition 2 (Syndrome Decoding Problem (SDP) [5]). *Let $H \in \mathbb{F}_2^{(n-k) \times n}$ be a parity-check matrix of a linear binary $[n, k]$ code, let $s \in \mathbb{F}_2^{n-k}$, and let w be a positive integer. The Syndrome Decoding Problem asks to find $e \in \mathbb{F}_2^n$ such that $He^T = s$ and $\text{wt}(e) \leq w$.*

The decision variant asks whether such an e exists; the optimisation variant minimises $\text{wt}(e)$ subject to $He^T = s$. SDP is NP-hard in its decision form [4] and is central in coding theory and cryptography.

Information Set Decoding (ISD)

ISD denotes a family of randomized exponential-time algorithms for SDP. The basic strategy is to guess an information set (positions assumed error-free), transform the instance accordingly, and recover an error vector more efficiently. The complexity depends primarily on (n, k) and the target weight w .

Starting from Prange [34], major improvements include Lee–Brickell [22], Leon [24], and Stern [38]. More recent variants use representations, nearest-neighbour search, and sieving [20,30]. ISD is the workhorse both for practical attacks and for concrete security estimates in code-based cryptography.

The link between SDP and ISD. ISD is best viewed as a toolbox for solving SDP instances: given (H, s, w) , it searches for an e satisfying $He^T = s$ with small weight. While SDP is NP-hard, ISD and its refinements provide the most effective known practical methods for many parameter ranges, and they underpin essentially all concrete attacks on code-based cryptosystems [6].

ISD algorithms. Prange [34] introduced information-set guessing; Leon and Stern [24,38] improved success probability using collisions and partial checks. Modern work further reduces complexity via representations and sieving; in particular, Johansson-style methods yield the fastest known classical decoders for many regimes [20,30].

Reduction from LWPM to SDP [26] Löndahl and Johansson showed that LWPM over $\mathbb{F}_2[x]$ can be reduced to a low-weight codeword problem in a structured linear code, and hence to SDP [27]. Concretely, for fixed $P(x)$ and degree bound n , the set of products $P(x)Q(x)$ with $\deg(Q) < n$ forms a linear code whose codewords are the coefficient vectors of these products. Finding a low-weight multiple of $P(x)$ is therefore equivalent to finding a low-weight codeword in this code, which can be formulated as an SDP instance.

This reduction is made explicit in Löndahl’s thesis [26], where LWPM is transformed into a syndrome decoding instance and solved with ISD-style methods (see, e.g., Algorithm 5 and Algorithm 10 in §6.2 of [26]). Thus, LWPM can be treated as a structured SDP instance, enabling the use of ISD algorithms (classical or sieving-based) as LWPM solvers.

Scope and focus of this work. The discussion above highlights that LWPM admits several algorithmic treatments, each naturally associated with different parameter regimes. In the remainder of this paper, we focus on the bounded-degree and low-weight regimes that arise most frequently in concrete cryptanalytic settings, in particular in correlation-style and key-recovery attacks on stream ciphers. These regimes are characterised by tight constraints on both the admissible degree of the multiple and its Hamming weight, and they are precisely the settings in which security claims are typically formulated.

Accordingly, our experimental study compares two classes of solvers that are well-suited to this regime: exact optimisation-based approaches (MILP and MAXSAT), which directly minimise the Hamming weight of $P(x)Q(x)$ under explicit constraints, and coding-theoretic approaches obtained by reducing LWPM to structured instances of the Syndrome Decoding Problem and solving them via Information Set Decoding (ISD).

We do not include LLL approaches on in our experimental comparison. As discussed in Section 2.2, such methods target a different optimisation criterion, provide neither optimality guarantees nor infeasibility certificates, and do not scale to the tightly constrained (n, w) regimes relevant for cryptanalysis. Consequently, LLL-based heuristics are not representative of the state of the art for evaluating concrete LWPM hardness in the settings considered here.

3 LWPM as an Optimisation Problem

In this section, we formalise the LWPM problem as an optimisation problem. We describe LPWM in a MILP (mixed integer linear programming) formulation and, using as a MAX-SAT problem, given in two MAX-SAT variations: XOR-MAXSAT and CNF-MAXSAT.

3.1 MILP formulation for bounded-weight LWPM

We describe a MILP formulation of LWPM using binary variables representing the coefficients of $Q(x)$ and of the product $P(x)Q(x)$. The convolution con-

straints are encoded using linear constraints with auxiliary variables, and the objective is to minimize the Hamming weight of the product.

We consider the following bounded-weight feasibility variant of LWPM. Given $P(x) \in \mathbb{F}_2[x]$ of degree n , a fixed target degree d for $Q(x)$, and a weight bound w' , the goal is to find a polynomial $Q(x) \in \mathbb{F}_2[x]$ with $\deg(Q) = d$ such that $\text{wt}(P(x)Q(x)) \leq w'$. This is a decision/feasibility version of LWPM in the simultaneously constrained (n, w) regime, with the additional restriction $\deg(Q) = d$.

Algorithm 1 LWPM to MILP (fixed $\deg(Q) = d$, bounded weight)

Require: $P(x) = \sum_{i=0}^n p_i x^i \in \mathbb{F}_2[x]$, degree n ; integer $d \geq 0$; weight bound w'
Ensure: A polynomial $Q(x) \in \mathbb{F}_2[x]$ with $\deg(Q) = d$ and $\text{wt}(PQ) \leq w'$, if it exists
1: Create binary variables $\{q_j\}_{j=0}^d$ and $\{r_k\}_{k=0}^{n+d}$
2: Create integer variables $\{u_k\}_{k=0}^{n+d}$ with $u_k \geq 0$
3: **for** $k = 0$ **to** $n + d$ **do**
4: Compute $S_k \leftarrow \{j \in \{0, \dots, d\} : 0 \leq k - j \leq n \wedge p_{k-j} = 1\}$
5: Add constraint $\sum_{j \in S_k} q_j - 2u_k - r_k = 0$
6: **end for**
7: Add constraint $q_d = 1$ {enforces $\deg(Q) = d$ }
8: Add constraint $\sum_{k=0}^{n+d} r_k \leq w'$
9: **(optional)** Set objective $\min \sum_{k=0}^{n+d} r_k$ {optimisation variant}
10: Call a MILP solver on the resulting model
11: Extract $q_0, \dots, q_d$ and output $Q(x) = \sum_{j=0}^d q_j x^j$

MILP variables. Let $P(x) = \sum_{i=0}^n p_i x^i$ with $p_i \in \{0, 1\}$ fixed. We introduce binary variables $q_j \in \{0, 1\}$ for $j = 0, \dots, d$ to represent $Q(x) = \sum_{j=0}^d q_j x^j$, and binary variables $r_k \in \{0, 1\}$ for $k = 0, \dots, n + d$ to represent the coefficients of $R(x) = P(x)Q(x)$. To encode parity constraints exactly, we also introduce integer variables $u_k \in \mathbb{Z}_{\geq 0}$.

For each $k \in \{0, \dots, n + d\}$ define the index set

$$S_k := \{j \in \{0, \dots, d\} \mid 0 \leq k - j \leq n \wedge p_{k-j} = 1\}.$$

Then the k -th product coefficient satisfies $r_k \equiv \sum_{j \in S_k} q_j \pmod{2}$.

MILP constraints. We enforce the parity exactly via

$$\sum_{j \in S_k} q_j - 2u_k - r_k = 0 \quad \text{for all } k = 0, \dots, n + d,$$

together with $r_k \in \{0, 1\}$ and $u_k \in \mathbb{Z}_{\geq 0}$. We also enforce $\deg(Q) = d$ by fixing $q_d = 1$. Finally, we enforce the bounded-weight condition by $\sum_{k=0}^{n+d} r_k \leq w'$. Optionally, the optimisation variant is obtained by minimising $\sum_{k=0}^{n+d} r_k$ instead of merely enforcing the inequality.

3.2 Encoding LWPM as XOR-MAXSAT and CNF-MAXSAT

We present now MAXSAT formulations from LWPM.

Let $P(x) = \sum_{i=0}^n p_i x^i \in \mathbb{F}_2[x]$ be given with $p_n = 1$ and let $d \geq 0$ be a fixed degree for $Q(x) = \sum_{j=0}^d q_j x^j$. Write $R(x) = P(x)Q(x) = \sum_{k=0}^{n+d} r_k x^k$. Over $\mathbb{F}_2$, the product coefficients satisfy

$$r_k = \bigoplus_{\substack{i+j=k \\ 0 \leq i \leq n \\ 0 \leq j \leq d}} (p_i \wedge q_j) = \bigoplus_{j \in S_k} q_j, \quad S_k := \{j \in \{0, \dots, d\} \mid 0 \leq k-j \leq n, p_{k-j} = 1\}.$$

Hence, each r_k is the XOR (parity) of a known subset of the q_j variables.

LWPM variant solved by the encodings. Two closely related encodings are targeted:

- (A) **LWPM to XOR-MAXSAT / MAX-XOR-SAT**, using native XOR constraints together with soft literals;
- (B) **LWPM to “plain” (W)CNF MAXSAT**, without native XOR constraints, relying instead on a standard XOR-to-CNF encoding.
 - **Optimisation (LWPM-OPT, fixed $\deg(Q) = d$):** minimise $\text{wt}(PQ) = \sum_{k=0}^{n+d} r_k$ subject to $\deg(Q) = d$. This is the regime implemented by encoding (A) above) via soft clauses on r_k .
 - **Decision/feasibility (LWPM-DEC, fixed $\deg(Q) = d$):** find Q with $\deg(Q) = d$ such that $\sum_{k=0}^{n+d} r_k \leq w$. This is implemented by encoding B above, via an “AtMost(w)” cardinality constraint on the r_k variables.

Encoding as XOR-MAXSAT (i.e., SAT with native XOR constraints)

We assume the solver supports XOR constraints natively, i.e., constraints of the form $\bigoplus_{\ell \in C} \ell = b$ where ℓ are literals and $b \in \{0, 1\}$. Such solvers are often referred to as XOR-SAT / XOR-MAXSAT solvers.

Remark. This encoding matches the reduction of LWPM to an optimisation problem in [39], over parity constraints: they capture multiplication in $\mathbb{F}_2[x]$, and the soft literals implement minimisation of Hamming weight. Note that each soft clause is the negated product-coefficient literal ($\neg r_k$): a MAXSAT solver maximises the total weight of satisfied soft clauses, so satisfying ($\neg r_k$) rewards $r_k = 0$ and the optimum minimises $\sum_k r_k$; soft clauses (r_k) would instead maximise the Hamming weight. Our experiment do not use soft clauses at all: as stated in Section 5 and Appendix E, they exercise the bounded-weight (hard-constraint) path of these algorithms, so no experimental result depends on this sign.

Encoding as plain MAXSAT without XOR (CNF/WCNF) Many MAXSAT solvers accept only CNF (or weighted CNF) and do not support XOR constraints natively. In this case, each XOR constraint must be compiled into CNF. A standard approach is to introduce auxiliary variables and use a Tseitin-style definitional encoding [41]. For long XORs, one can chain binary XORs:

Algorithm 2 LWPM to XOR-MAXSAT

Require: $P(x) \in \mathbb{F}_2[x]$ of degree n ; fixed d ; (optional) weight bound w
Ensure: $Q(x) \in \mathbb{F}_2[x]$ with $\deg(Q) = d$ minimising $\text{wt}(PQ)$

- 1: Create Boolean variables $q_0, \dots, q_d$ and $r_0, \dots, r_{n+d}$
- 2: Add hard constraint $q_d = 1$ {enforces $\deg(Q) = d$ }
- 3: **for** $k = 0$ **to** $n + d$ **do**
- 4: Compute $S_k := \{j \in \{0, \dots, d\} : 0 \leq k - j \leq n \wedge p_{k-j} = 1\}$
- 5: **if** $S_k = \emptyset$ **then**
- 6: Add hard clause $\neg r_k$
- 7: **else**
- 8: Add hard XOR constraint $r_k \oplus \bigoplus_{j \in S_k} q_j = 0$
- 9: **end if**
- 10: **end for**
- 11: **for** $k = 0$ **to** $n + d$ **do**
- 12: Add soft literal $\neg r_k$ (i.e., the soft clause $(\neg r_k)$) with weight 1 {minimise $\sum_k r_k$ }
- 13: **end for**
- 14: **(optional)** Add hard constraint $\sum_{k=0}^{n+d} r_k \leq w$ {feasibility variant}
- 15: Call an XOR-MAXSAT solver and extract an optimal assignment for (q_j)
- 16: Output $Q(x) = \sum_{j=0}^d q_j x^j$

Algorithm 3 LWPM to CNF-MAXSAT (i.e., no XOR support in SAT)

Require: $P(x) \in \mathbb{F}_2[x]$ of degree n ; fixed d ; either bound w (decision) or optimisation
Ensure: $Q(x) \in \mathbb{F}_2[x]$ satisfying the requested variant

- 1: Create Boolean variables $q_0, \dots, q_d$ and $r_0, \dots, r_{n+d}$
- 2: Add hard clause (q_d) {enforces $\deg(Q) = d$ }
- 3: **for** $k = 0$ **to** $n + d$ **do**
- 4: Compute $S_k := \{j : 0 \leq k - j \leq n \wedge p_{k-j} = 1\}$
- 5: **if** $S_k = \emptyset$ **then**
- 6: Add hard clause $(\neg r_k)$
- 7: **else**
- 8: Let $A_k = \{q_j : j \in S_k\}$ and encode $r_k \leftrightarrow \bigoplus A_k$ into CNF:
- 9: Introduce fresh variables and chain binary XORs using Tseitin clauses [41]
- 10: **end if**
- 11: **end for**
- 12: **if Decision variant then**
- 13: Encode the cardinality constraint $\sum_{k=0}^{n+d} r_k \leq w$ into CNF [37]
- 14: Call a SAT solver (or hard-CNF MAXSAT solver) and extract (q_j) if satisfiable
- 15: **else if Optimisation variant then**
- 16: **for** $k = 0$ **to** $n + d$ **do**
- 17: Add soft unit clause $(\neg r_k)$ with weight 1 {minimise $\sum_k r_k$ }
- 18: **end for**
- 19: Call a WCNF MAXSAT solver and extract an optimal assignment for (q_j)
- 20: **end if**
- 21: Output $Q(x) = \sum_{j=0}^d q_j x^j$

introduce fresh variables $y_1, \dots, y_m$ such that $y_1 \leftrightarrow (a_1 \oplus a_2)$, $y_2 \leftrightarrow (y_1 \oplus a_3)$, $\dots$, and finally $r_k \leftrightarrow (y_{m-1} \oplus a_m)$, where $a_1, \dots, a_m$ are the literals in the XOR.

Each binary XOR equivalence $c \leftrightarrow (a \oplus b)$ can be expressed by the following 4 CNF clauses: $(\neg a \vee \neg b \vee \neg c) \wedge (\neg a \vee b \vee c) \wedge (a \vee \neg b \vee c) \wedge (a \vee b \vee \neg c)$.

Decision (bounded weight). To enforce $\sum_k r_k \leq w$ in CNF, one may use a standard cardinality encoding, e.g., sequential counters [37] or other polynomial encodings.

Optimisation (minimise weight). To minimise $\sum_k r_k$, add each $\neg r_k$ as a unit soft clause of weight 1 in WCNF.

Discussion. Encoding LWPM using native XOR constraints preserves the linear structure of the multiplication constraints and is typically far more compact than compiling XOR to CNF. When XOR is compiled away, the CNF size grows with the number and length of parity constraints, and performance depends heavily on the chosen XOR-to-CNF and cardinality encodings [41,37].

4 Solving LWPM via Classical ISD

This section formalises an ISD-based pipeline for LWPM. We

- (i) use the reduction of an LWPM instance to a structured low-weight codeword problem equivalently, a SDP instance with zero syndrome, following the coding-theoretic approach by Loendahl [26], and
- (ii) solve the resulting SDP instance using Stern’s canonical information-set decoding algorithm [38].

4.1 From LWPM to a structured SDP instance

Let $P(x) = \sum_{i=0}^n p_i x^i \in \mathbb{F}_2[x]$ be a non-zero polynomial of degree n , and fix a degree bound d for $Q(x) = \sum_{j=0}^d q_j x^j \in \mathbb{F}_2[x]$ (often we additionally enforce $q_d = 1$ to ensure $\deg(Q) = d$). Define $R(x) = P(x)Q(x)$ and view it as a binary vector $r \in \mathbb{F}_2^n$ of length $t := n + d + 1$ given by its coefficients.

The key observation exploited in [27] is that the set of all such products $R = P \cdot Q$ (for $\deg(Q) \leq d$) forms a linear code $\mathcal{C} \subseteq \mathbb{F}_2^n$ with a highly structured generator description induced by polynomial multiplication. Consequently, finding a low-weight multiple is equivalent to finding a low-weight codeword in $\mathcal{C}$.

Generator representation. Let $G \in \mathbb{F}_2^{k \times t}$ be a generator matrix for $\mathcal{C}$, where typically $k = d + 1$ (one row per monomial multiplier x^j), and the j -th row is the coefficient vector of $x^j P(x)$ truncated to length n (non-cyclic Toeplitz “convolution” model) or taken modulo $x^n - 1$ (cyclic/circulant model), depending on the exact LWPM variant studied [27]. Then every candidate product satisfies $r = uG$ for some $u \in \mathbb{F}_2^k$, and the LWPM condition $\text{wt}(PQ) \leq w$ becomes $\text{wt}(r) \leq w$.

SDP view. Let $H \in \mathbb{F}_2^{(t-k) \times t}$ be any parity-check matrix for $\mathcal{C}$ (i.e., $HG^\top = 0$). Then $r \in \mathcal{C}$ iff $Hr^\top = 0$. Therefore, low-weight codeword search in $\mathcal{C}$ is equivalent to the SDP instance with zero syndrome:

$$\text{find } r \in \mathbb{F}_2^n \setminus \{0\} \text{ such that } Hr^\top = 0 \text{ and } \text{wt}(r) \leq w.$$

This is precisely the form addressed by ISD algorithms such as Stern’s [38].

4.2 Stern’s ISD for low-weight codeword search

Stern’s algorithm [38] (given below for reference) is a probabilistic improvement over earlier ISD methods [34,24] for finding a vector of small Hamming weight satisfying a linear constraint (here, $Hr^\top = 0$). In its codeword-search variant, it repeatedly: (i) randomises the instance by permuting coordinates (columns), (ii) puts the code description in a convenient form (e.g., systematic form), (iii) chooses a window B of coordinates in which collisions are forced, and (iv) uses a meet-in-the-middle collision step to build candidate low-weight codewords, which are then filtered by a weight test.

Algorithm 4 STERN’S SDP-SOLVER [38] (PRESENTED IN OUR TERMINOLOGY)

Require: Parity-check matrix $H \in \mathbb{F}_2^{(n-k) \times n}$; weight target w ; parameters: p (subset size), $|B|$ (window size), L (list cap), T (iterations)

Ensure: A non-zero $r \in \mathbb{F}_2^n$ with $Hr^\top = 0$ and $\text{wt}(r) \leq w$, if found

```

1: for  $t = 1$  to  $T$  do
2:   Sample a random column permutation  $\pi \in S_n$  and set  $H' \leftarrow H \cdot \pi$ 
3:   Compute a systematic-like form of  $H'$  (or equivalently of a dual generator),
      recording  $\pi$ 
4:   Choose a window  $B \subseteq \{1, \dots, n\}$  of size  $|B|$ 
5:   Split the (implicit) basis vectors used for combinations into two sets  $R_1, R_2$ 
6:   Build a hash table  $\mathcal{L}_1$ : for up to  $L$  sampled  $p$ -subsets  $I \subseteq R_1$ , store the projection
      of the partial sum onto  $B$ 
7:   For up to  $L$  sampled  $p$ -subsets  $J \subseteq R_2$ :
8:     compute the same projection key on  $B$  and look for collisions in  $\mathcal{L}_1$ 
9:     for each collision, form a candidate  $r'$  (by XORing the two partial sums)
10:    unpermute  $r \leftarrow \pi^{-1}(r')$  and test  $Hr^\top = 0$  and  $\text{wt}(r) \leq w$ 
11:    if test passes and  $r \neq 0$  then
12:      return  $r$ 
13:    end if
14: end for
15: return fail

```

Why Stern’s algorithm. Among classical ISD algorithms, Stern’s algorithm [38] is widely regarded as a canonical representative. It generalises and subsumes earlier ISD variants such as Prange’s algorithm [34], the Lee–Brickell improvement [22], and Leon’s collision-based refinements [24], while retaining the same underlying information-set framework. As a result, Stern’s algorithm is commonly used as the reference point for “classical” ISD in both theoretical analyses and practical comparisons (e.g., [27,6]).

For this reason, we use Stern’s algorithm as our main ISD-based case-study, and later in the paper, as one of our representative classical ISD solver when comparing optimisation-based approaches with ISD-based methods.

Note: In Appendices A and D, to expand the significance of our results, we also study, in the same fashion, the BJMM algorithm [2], the representation-based ISD variant on which concrete ISD cost estimates are nowadays based; it is run under the same protocol and reported alongside Stern in Section 5 and Appendix D. Nearest-neighbour and sieving-based variants are discussed

there too; their advantage only materialises at code lengths beyond those of our instances.

Note 2: As Appendices C and D (Tables 5–10) show, switching the ISD variant does not change the picture. BJMM is consistently faster per attempt than Stern (median wall-times lower by a factor of roughly 3 to 30× across our parameter points), yet the two solve nearly the same sets of instances within the time budget, and they fail together: at the parameter points where Stern solves no instance within the limit (e.g. the block instances of (150, 40, 10) and (250, 70, 10) in the fixed-weight variant), BJMM solves none either, while the optimisation solvers solve all 250 in seconds. Likewise, on the infeasible instances of Section 5.7 ($w < 10$ at (200, 60)), both ISD variants can only exhaust their iteration budget inconclusively, whereas the optimisation solvers return a certificate of infeasibility in well under a second. This is expected: the gain of the representation technique grows with the number of representations $R = \binom{p}{p/2} \binom{k-p}{\varepsilon}$, which is small in the bounded-degree, low-weight regime studied here. Indeed, BJMM explores essentially the same search space as Stern, only with cheaper iterations. Since BJMM confirms rather than changes the conclusions drawn with Stern, we keep Stern as the ISD representative in the main text, which thus remains self-contained, and defer BJMM to Appendixes B/E as a robustness check on the choice of ISD variant.

4.3 From LWPM to SDP & Classic ISD Afterwards

Algorithm 5 LWPM to SDP, Solved via Stern’s ISD

Require: Polynomial $P(x) \in \mathbb{F}_2[x]$ of degree n ; fixed multiplier degree d ; target weight w ; choice $\text{model} \in \{\text{Toeplitz}, \text{cyclic}\}$; Stern parameters $(p, |B|, L, T)$
Ensure: A polynomial $Q(x)$ with $\deg(Q) = d$ such that $\text{wt}(PQ) \leq w$, if found
1: $t \leftarrow n + d + 1$; $k \leftarrow d + 1$
2: **(Reduction [26])** Build generator matrix $G \in \mathbb{F}_2^{k \times t}$ for the product code $\mathcal{C}$ induced by $P(x)$ and model [27]
3: Compute a parity-check matrix $H \in \mathbb{F}_2^{(t-k) \times t}$ for $\mathcal{C}$ (so $HG^\top = 0$)
4: **(SDP instance)** Set syndrome $s \leftarrow 0 \in \mathbb{F}_2^{t-k}$
5: Run $\text{STERN-CODEWORD-SEARCH}(H, w; p, |B|, L, T)$ to find $r \neq 0$ with $Hr^\top = s$ and $\text{wt}(r) \leq w$
6: **if** no r found **then**
7: **return fail**
8: **end if**
9: Recover an information vector $u \in \mathbb{F}_2^k$ such that $r = uG$ (e.g., by keeping track of the systematic transformation/permutation used internally)
10: Interpret $u = (q_0, \dots, q_d)$ as coefficients of $Q(x) = \sum_{j=0}^d q_j x^j$
11: **return** $Q(x)$

Comment on modelling choices. The code construction step (Toeplitz/non-cyclic versus cyclic/circulant) determines which family of LWPM instances is being attacked; both arise in the literature, depending on whether multiplication is interpreted as plain convolution or as convolution modulo $x^n - 1$ in a cyclic code

setting [27]. Our implementation-level experiments can therefore switch model while keeping the ISD layer identical.

5 Experimental Methodology

This section presents a unified and streamlined experimental evaluation of the algorithmic approaches studied in this work and mentioned thus far.

Note: All our code and our evaluation data is found at [21].

Experimental platform. All experiments were conducted on the same machine to ensure comparability. The hardware and system configuration was a server with two Intel Xeon E5-2667 V3 at 3.20GHz CPUs (8 cores, 16 threads each) and 378GB of RAM. Wall-clock time, CPU time, and peak memory consumption were measured at the operating-system level. All solvers were run in the default configuration: CPLEX was invoked with the default/automatic number of threads (so it may use several cores), and PySAT, IGMxHS, and our ISD implementations are single-threaded by construction. This asymmetry can only favour CPLEX, and it means CPLEX’s CPU time can exceed its wall-time; all cross-solver comparisons in this paper are therefore drawn on wall-clock time. Appendix E restates these settings and discusses the wider implementation differences between the solvers.

5.1 Instance generation

We evaluate the solvers on several families of LWPM instances designed to capture both cryptographically relevant and structurally informative cases.

Random sparse polynomials. Random instances are generated by fixing the degree n of $P(x)$ and selecting its support uniformly at random with a prescribed Hamming weight $w_P \ll n$. For larger degrees, we typically set $w_P \approx n/20$. These instances model the generic case commonly assumed in cryptanalytic analyses. The concrete generation procedure is given in Appendix B.

Non-random polynomials. To probe solver sensitivity to algebraic structure, we also consider: structured polynomials with a contiguous block of 1-coefficients a variable length. Such instances, while less random, are useful for exposing the strengths and weaknesses of different algorithmic paradigms. The exact generation procedures are deferred to Appendix B.

Parameter regimes. Experiments cover both fixed-weight regimes (exact target weight w) and bounded-weight regimes ($\text{wt}(P(x)Q(x)) \leq w$), reflecting typical cryptanalytic use cases. We consider both medium-scale instances, with degrees ranging up to $n = 400$, and multipliers $Q(x)$ of degree up to $d = 100$; more details follow in Section 5.6.

5.2 Solvers and encodings

Each LWPM instance is solved using the following approaches:

- **MILP**. The formulation from Section 3.1, solved using a generic mixed-integer linear programming solver.
Note: Standard open-source linear programming (LP) solvers, such as GLPK, present limited scalability in modelling XOR constraints, which we need here [3]: XOR modelling needs auxiliary variables and the branch-and-bound tree grows exponentially [42], whereas CPLEX, with its pre-solve and cutting mechanisms, scales far better on such models [33,25,1]. We therefore use CPLEX.
- **XOR-SAT / MAXSAT (PySAT)**. A native XOR encoding of the polynomial multiplication constraints, solved using CryptoMiniSat through the PySAT interface.
- **CNF MAXSAT (IGMaxHS)**. A pure CNF encoding obtained via XOR-to-CNF translation, combined with sequential counter encodings for cardinality constraints.
- **ISD (Stern, BJMM)**. ISD applied after reduction of LWPM to a structured SDP instance, as described in Section 4; the representation-based BJMM variant (Appendix A) is run under the same protocol.

Solver configuration. Every solver is used in its default, off-the-shelf configuration: no CPLEX parameter (MIP emphasis, heuristic frequency, cut generation) was tuned, no solver-specific pre-processing was added, and IGMaxHS exposes no tuning parameters at all. In all benchmarks, the three optimisation-based solvers are run on the bounded-weight path of Algorithms 1–3, i.e., with hard cardinality constraints (exactly w in the fixed- w experiments; $1 \leq \text{wt}(PQ) \leq w$ in the variable- w experiments) and no soft clauses or minimising objective; a returned model is a multiple of weight $\leq w$, and an UNSAT/infeasible answer is a proof that none exists in the stated range. This is deliberate. The question we ask is whether generic, unoptimised SAT/MILP solving can compete with specialised LWPM solving, so tuning the optimisation side would bias the comparison in its favour. Appendix E lists the configurations used and reports a small tuning experiment on CPLEX.

5.3 Evaluation metrics

We report the following metrics: 1. Time to first solution: wall-clock time to a valid LWPM solution. 2. Solution quality: the Hamming weight of $P(x)Q(x)$. 3. Optimality status: for optimisation-based solvers, whether the solution is proven optimal or merely feasible. 4. Resource usage: peak memory consumption and total CPU time.

5.4 Overall experimental results

We ran optimisation solvers (i.e., MILP - in Sage/Python, PySAT - in Python), IGMaxHS - in C/Python) and an ISD solver (Stern and BJMM - in Sage/Python). For every solver, we used the baseline implementation without altering the internal heuristics, cutting strategies, or preprocessing routines. The comparison focuses, therefore, exclusively on the behavior of the original solvers.

In the summary of our experiments shown in this section, we show eight representative parameter settings in which we ran the above solvers, i.e.,:

$(n, d, w) = (50, 10, 8)$, $(n, d, w) = (100, 20, 8)$, $(n, d, w) = (150, 40, 10)$, $(n, d, w) = (200, 60, 10)$, $(n, d, w) = (250, 70, 10)$, $(n, d, w) = (300, 80, 10)$, $(n, d, w) = (350, 90, 10)$, and $(n, d, w) = (400, 100, 10)$.

For each setting, we evaluate variants with fixed w , bounded w , and variable n . For every considered parameter tuple (n, d, w) , we generate and evaluate a benchmark set of 250 independent instances to ensure statistically robust evaluation across configurations.

5.5 LWPM Solved as Optimisation Problem.

Across random and non-random regimes, the three optimisation-based solvers return multiples of the same weight; where they differ is in running time and memory, and, when w is left free (Appendix C), in how far below the bound they get, with IGMaXHS returning the lowest average weight.

Table 2 summarizes wall-clock time, peak memory usage, and CPU time for sparse random instances, under the configuration $(n, d, w) = (400, 100, 10)$, where n and w vary up to the maximum limit of each of them, while d remains fixed. We leave empty entries when no solution was obtained.

All evaluated solvers produced identical average, median, and maximum weights (namely 10.0; the weight table is moved to Appendix C to save space), indicating that the compared solvers generate equivalent-quality solutions.

The CPU-time columns of Table 2 show that The MILP and PySAT formulations achieved the lowest average CPU time, but PySAT exhibited substantially larger memory usage, as we can observe in its peak-memory columns. BJMM achieves, on average, a slightly higher CPU time than PySAT and MILP. Both BJMM and Stern exhibit much higher memory utilization than either of the optimisation-based solvers.

Turning to wall-time, The MILP formulation has the best overall performance, being consistent with the CPU-time measurements. PySAT showed the largest maximum wall-time. IGMaXHS provided an intermediate trade-off between runtime and memory consumption. BJMM remains competitive with PySAT, while Stern incurs substantially higher wall-clock times.

In random regimes, MILP solvers exhibit low runtime and memory usage. IGMaXHS frequently achieves the best weights but is worse in runtime and memory consumption than PySAT and MILP-CPLEX solver. Although it is often the most efficient in terms of CPU, PySat is the most expensive in terms of peak memory. In the context of block polynomials, MILP maintains the previously mentioned behavior for small parameters, while IGMaX is the most time-consuming. PySat remains the largest memory consumer when we refer to optimisation-based solvers. Stern is the most expensive method in this regime, while BJMM, although considerably more efficient than Stern, requires much more memory than MILP and IGMaXHS. Overall, the ISD-based methods are substantially more resource-intensive, especially in terms of memory consumption and CPU time.

Table 2: Random regime, $(n, d, w) = (400, 100, 10)$ (v : varies up to the value shown; f : fixed): CPU time, peak memory and wall-time over 250 instances. (All solvers returned weight exactly 10; the weight table is now in Appendix C.)

No.	n	d	w	Solver	CPU time (s)			Peak memory (MiB)			Wall-time (s)		
					Max	Avg	Med	Max	Avg	Med	Max	Avg	Med
1	400 ^v	100 ^f	10 ^v	MILP	0.26	0.2	0.21	0.2	0.19	0.19	0.36	0.24	0.25
2	400 ^v	100 ^f	10 ^v	IGMaxHS	0.64	0.28	0.28	3.25	0.37	0.36	0.64	0.29	0.28
3	400 ^v	100 ^f	10 ^v	PySat	2.14	0.2	0.11	76.41	73.41	73.71	2.58	0.64	0.53
4	400 ^v	100 ^f	10 ^v	Stern (250/250)	18.57	3.1	2.68	384.23	279.01	263.61	18.6	3.12	2.7
5	400 ^v	100 ^f	10 ^v	BJMM (250/250)	2.11	0.44	0.36	212.14	208.94	208.71	2.13	0.46	0.38

In non-random regimes, similar results could be observed, as can be seen in Table 3 and Appendix C, but with the following observations.

Under the configuration $(n, d, w) = (300, 80, 10)$, with n and w varying up to the maximum limit of each of them, while d remains fixed, all optimisation-based solvers again produced identical maximum, average, and median product weights (all 10.0), thus confirming the functional consistency of the compared methods on the tested instances. The ISD methods failed to solve any of the instances within the set time limit.

The CPU-time columns of Table 3 present the CPU-time consumption of the evaluated approaches. PySAT achieved the best performance, while MILP formulation remained competitive, and IGMaxHS showed a much higher consumption. Stern exceeded the time limit for all instances, which led to no valid CPU-time statistics. BJMM completed its executions within the imposed time limit with higher CPU consumption than the optimisation-based solvers, but still failed to find a solution for any instance.

The peak memory consumption columns confirm the tendency previously observed. The MILP formulation is consistent with the previous experiments, requiring the smallest memory usage. Again, PySAT evidenced a substantially higher memory usage, while IGMaxHS provided the trade-off between execution time and memory consumption. Although BJMM is more efficient than Stern, both have much higher memory consumption than the other three solvers.

The wall-time columns respect the premises from the CPU measurements. PySAT achieved the best performance, followed by the MILP formulation. IGMaxHS remained the slowest among the evaluated solvers, while BJMM is the most time-consuming.

On the Best Performing LPWM-Solvers

For the best LPWM solvers, in Figure 1, we have also systematised some more results linked to MILP and MAXSAT-solving, in a visual way. We see that although the results obtained by MILP are better in terms of execution time and peak memory, MILP’s CPU consumption has an irregular distribution.

Figure 1 summarises, in the first part, the wall-time under the configuration $(n, d, w) = (400, 100, 10)$, where n and w vary up to the maximum limit

Table 3: Non-random (block) regime, $(n, d, w) = (300, 80, 10)$: CPU time, peak memory and wall-time over 250 instances (all solvers returned weight exactly 10). ‘TO’ denotes the 120 s time limit was exceeded. ‘N/A’ denotes the statistic is unavailable because no solution was found within the time limit imposed.

No.	n	d	w	Solver	CPU time (s)			Peak memory (MiB)			Wall-time (s)		
					Max	Avg	Med	Max	Avg	Med	Max	Avg	Med
1	300 ^v	80 ^f	10 ^f	MILP	7.73	0.38	0.11	0.21	0.14	0.14	7.88	0.40	0.14
2	300 ^v	80 ^f	10 ^f	IGMaxHS	13.97	6.97	5.63	8.37	5.79	5.78	14.03	6.99	5.64
3	300 ^v	80 ^f	10 ^f	PySat	0.72	0.10	0.03	75.86	73.37	73.92	2.04	0.51	0.43
3	300 ^v	80 ^f	10 ^f	Stern (0/250)	N/A	N/A	N/A	517.93	505.62	505.55	TO	TO	TO
5	300 ^v	80 ^f	10 ^f	BJMM (0/250)	48.57	23.6	22.57	200.16	198.29	198.38	48.58	23.62	22.59

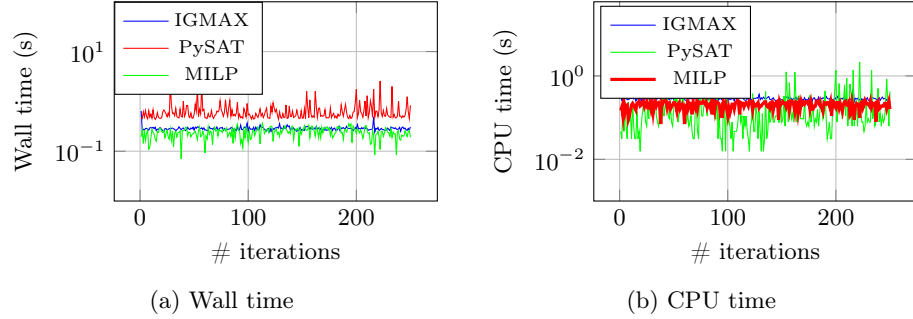


Fig. 1: Comparison among the most competitive solvers for $n = 400$, $d = 100$, $w = 10$, weightP=10 ($P(x)$ of degree $\leq n$, $Q(x)$ of fixed degree d , weight $\leq w$).

of each of them, while d remains fixed. As per Table 2, PySAT presents the highest running time, followed by IGMaxHS and the MILP formulation, which obtained the best performance. In the second part, Figure 1 describes CPU time consumption and reinforces the previously mentioned conclusions. The MILP formulation is consistent with the statistics reported in Table 2 and exhibits a stable runtime behavior with a relatively low variance across the 250 instances used. IGMaxHS had slightly higher wall times than MILP, but still comparatively consistent. Conversely, PySAT demonstrates the greatest fluctuation, with several pronounced spikes.

Regarding the peak memory consumption, the assumptions in Table 2 are maintained. PySat shows the highest consumption, while IGMaxHS and MILP run close, with MILP having the lowest consumption.

Also, since the most competitive solvers are the optimisation-based ones (not the ISD-based one), we include vastly more experimental data/results on them in Appendix C. The Appendix C tables are organized according to the considered evaluation regimes. Tables 5 and 6 summarize the experiments performed on the random regime, while Table 7 presents the corresponding results for the non-random regime. The new experiments requested during review (varying w at fixed (n, d) , ISD at further parameter points, and real feedback polynomials) are summarised next and detailed in Appendix D.

5.6 LWPM Solved as an SDP Problem, with ISD

For the ISD approach via Stern, a total of 250 benchmark instances for $(n, d, w) = (200, 60, 10)$ were evaluated with a time limit of 120 seconds, the same limit imposed on all solvers in Table 4.

For a weight $w = 10$, all solvers successfully solve all 250 instances, with MILP achieving the lowest median wall-time. For every weight below 10, it turns out that no solution exists on any of the 250 instances, and the two solver families report this fundamentally differently. The exact solvers (MILP, IGMMaxHS, PySAT) terminate with a proof that no multiple of weight $\leq w$ exists, in median times below a second. Stern and BJMM cannot certify non-existence: they run until their iteration budget is exhausted and stop without an answer (median ≈ 95 s and ≈ 3 s respectively), which is compatible both with infeasibility and with a missed solution.

For instances that exceed this parameter regime, the proportion of instances that cannot be resolved due to the imposed time limitation increases considerably, especially if the weight is fixed ; Table 9 in Appendix D reports Stern and BJMM at $(100, 20, 10)$, $(250, 70, 10)$, $(300, 80, 10)$ in the random regime, and $(100, 20, 8)$ and $(150, 40, 10)$ the block regime, under the same 120 s limit.

5.7 Varying the Target Weight at Fixed (n, d)

To compare all four solvers on the same instances and under the same time limit, we fix $(n, d) = (200, 60)$ and decrease the weight bound w from 10 down to 6, on the 250 random instances of Section 5.6. Table 4 reports, per solver and per w , the number of instances for which a multiple of weight $\leq w$ was returned within 120 s and the median wall-time, where the time averages over different events, as follows. For $w = 10$, all solvers return a multiple on all instances, and the time is the median time to a solution. For $w \leq 9$, no multiple of weight $\leq w$ exists on any instance: the exact solvers (MILP, IGMMaxHS, PySAT) prove infeasibility, and their entry gives the median time of that proof; Stern and BJMM stop without an answer once their iteration budget is exhausted, and their entry gives the median time to that inconclusive stop. Thus, as w decreases, exact solvers keep giving a definite answer within tenths of a second, while the ISD baselines spend orders of magnitude longer and settle nothing, and this asymmetry, i.e., the ability to certify non-existence, is itself one of the main practical differences between the two paradigms.

Real feedback polynomials. Random and block polynomials are useful as controlled families, but the cryptographically relevant instances are feedback polynomials of deployed LFSRs. Appendix D.3 runs all solvers on the degree-53 polynomial used as the running example by Didier and Laigle-Chapuy [10], and on the register polynomials of the A5/1 and Bluetooth E0 ciphers, with d up to several hundred. ISD-based methods, especially BJMM, can reach lower-weight multiples in some cases while remaining competitive in runtime.

Table 4: Random regime, $(n, d) = (200, 60)$, 250 instances, 120 s limit per instance: number of instances solved (median wall-time in seconds) for each target weight w ; for $w \leq 9$ no multiple of weight $\leq w$ exists, and the time in parentheses is the median time to prove infeasibility (MILP, IGMaHS, PySAT), respectively to stop without an answer at budget exhaustion (Stern, BJMM).

Solver	$w = 10$	$w = 9$	$w = 8$	$w = 7$	$w = 6$
MILP (CPLEX)	250 (0.08)	0 (0.32)	0 (0.2)	0 (0.14)	0 (0.1)
IGMaHS	250 (0.13)	0 (0.1)	0 (0.09)	0 (0.08)	0 (0.06)
PySAT	250 (0.47)	0 (0.57)	0 (0.55)	0 (0.48)	0 (0.49)
ISD (Stern)	250 (0.42)	0 (95.35)	0 (94.97)	0 (94.57)	0 (95.54)
ISD (BJMM)	250 (0.17)	0 (3.02)	0 (3.02)	0 (3.04)	0 (3.01)

Overall View on the Experiments. Taken together, the results show that, in the bounded-degree, low-weight regimes relevant to cryptanalysis, off-the-shelf optimisation solvers (MILP and MAXSAT) solve a larger fraction of instances within the imposed time limit than the ISD baselines, with lower and far less dispersed running times, and on the random and block families never with a worse weight, besides being the only ones able to certify that no multiple of a given weight exists (Table 4); on the real feedback polynomials of Appendix D.3, anytime ISD reached lower weights within the same budget (Table 10). We do not claim that ISD is dominated: it remains the reference in large, random, high-weight settings, which lie outside our experiments, and our ISD implementations are research-grade Python code, whereas CPLEX is a commercial product (see Appendix E for how we control for this). Our message is that generic optimisation solving is a credible competitor of specialised LWPM solving in these regimes, so security assessments of LWPM-based constructions should consider both.

6 Conclusions

We presented an experimental study of the Low-Weight Polynomial Multiple (LWPM) problem, comparing coding-theoretic approaches based on Information Set Decoding (ISD) with exact optimisation methods using MILP and MAXSAT. Our results show that, in bounded-degree, low-weight regimes relevant to cryptanalysis, and sometimes even on feedback polynomials of published LFSR-based ciphers, off-the-shelf optimisation solvers often solve a larger fraction of instances within a fixed time budget than ISD-based solving (Stern and BJMM), with lower and more predictable running times, and with the same or very comparable weights whenever both succeed.

This work shows that, for LWPM, exact optimisation offers a credible alternative that should be considered alongside coding-theoretic methods. We kept the two paradigms deliberately separate; combining them is the natural next step (Appendix F). Future work includes studying larger parameter ranges, nearest-neighbour and sieving-based ISD variants, solver tuning, and hybrid optimisation-decoding techniques.

References

1. Basu, A., Conforti, M., Summa, M.D., Jiang, H.: Complexity of branch-and-bound and cutting planes in mixed-integer optimization—ii. *Combinatorica* **42**(Suppl 1), 971–996 (2022)
2. Becker, A., Joux, A., May, A., Meurer, A.: Decoding Random Binary Linear Codes in $2^{n/20}$: How $1 + 1 = 0$ Improves Information Set Decoding. In: *EUROCRYPT 2012*. pp. 520–536. Springer (2012). https://doi.org/10.1007/978-3-642-29011-4_31, LNCS 7237
3. Bellini, E., De Piccoli, A., Formenti, M., Gerault, D., Huynh, P., Pelizzola, S., Polese, S., Visconti, A.: Differential cryptanalysis with sat, smt, milp, and cp: a detailed comparison for bit-oriented primitives. In: *International Conference on cryptology and network security*. pp. 268–292. Springer (2023)
4. Berlekamp, E., McEliece, R., van Tilborg, H.: On the inherent intractability of certain coding problems. *IEEE Transactions on Information Theory* **24**(3), 384–386 (1978)
5. Berlekamp, E., McEliece, R., Van Tilborg, H.: On the inherent intractability of certain coding problems (corresp.). *IEEE Transactions on Information theory* **24**(3), 384–386 (1978)
6. Bernstein, D.J., Lange, T., Peters, C.: Attacking and defending the mceliece cryptosystem. *Post-Quantum Cryptography* (2011)
7. Biryukov, A., Shamir, A., Wagner, D.: Real time cryptanalysis of A5/1 on a PC. In: *Fast Software Encryption (FSE 2000)*. LNCS, vol. 1978, pp. 1–18. Springer (2001)
8. Bluetooth SIG: Bluetooth core specification, vol. 2, part h: Security specification (E0 encryption). <https://www.bluetooth.com/specifications/specs/> (2023), accessed August 2026
9. Chose, P., Joux, A., Mitton, M.: Fast correlation attacks: An algorithmic point of view. *EUROCRYPT 2002* (2002)
10. Didier, F., Laigle-Chapuy, Y.: Finding low-weight polynomial multiples using discrete logarithm. In: *IEEE International Symposium on Information Theory (ISIT 2007)*. pp. 1036–1040. IEEE (2007)
11. ECRYPT Stream Cipher Project: eSTREAM: The ECRYPT Stream Cipher Project (2008), <https://competitions.cr.yp.to/esteam.html>, final portfolio of stream ciphers, accessed June 2026
12. El Aimani, L., von zur Gathen, J.: Finding low weight polynomial multiples using lattices. *IACR Cryptology ePrint Archive* (2007)
13. Esser, A., Bellini, E.: Syndrome decoding estimator. In: *Public-Key Cryptography – PKC 2022*. LNCS, vol. 13177, pp. 112–141. Springer (2022). https://doi.org/10.1007/978-3-030-97121-2_5
14. ETSI, 3GPP: Specification of the 3GPP Confidentiality and Integrity Algorithms EEA3 & EIA3; Document 2: ZUC Specification. ETSI Technical Specification TS 135 222, European Telecommunications Standards Institute (2024), https://www.etsi.org/deliver/etsi_ts/135200_135299/135222/18.00.00_60/ts_135222v180000p.pdf, 3GPP TS 35.222 Release 18, accessed June 2026
15. ETSI, 3GPP: Specification of the 3GPP Confidentiality and Integrity Algorithms UEA2 & UIA2; Document 2: SNOW 3G Specification. Tech. Rep. TS 135 216, European Telecommunications Standards Institute (2024), https://www.etsi.org/deliver/etsi_ts/135200_135299/135216/18.00.00_60/ts_135216v180000p.pdf, version 18.0.0, accessed June 2026

16. Golić, J.D.: Computation of low-weight parity-check polynomials. *Electronics Letters* **32**(21), 1979–1980 (1996)
17. Hell, M., Johansson, T., Maximov, A., Meier, W., Sönnerup, J., Yoshida, H.: Grain-128AEADv2: A Lightweight AEAD Stream Cipher. <https://csrc.nist.gov/CSRC/media/Projects/lightweight-cryptography/documents/finalist-round/updated-spec-doc/grain-128aead-spec-final.pdf> (2021), nIST Lightweight Cryptography Project
18. IBM: IBM ILOG CPLEX optimization studio: List of CPLEX parameters. In: <https://www.ibm.com/docs/en/cofz/12.10.0?topic=cplex-list-parameters> (2020)
19. ISO/IEC: Information technology — Automatic identification and data capture techniques — Part 13: Crypto suite Grain-128A security services for air interface communications. <https://www.iso.org/standard/88956.html> (2026), ISO/IEC 29167-13:2026
20. Johansson, T.: Information set decoding with nearest neighbor search. In: *Information Theory Workshop (ITW)*. pp. 1–5. IEEE (2013)
21. Lazarescu, S., Boureanu, I., Dragan, C.: Code and Data for Paper on LWPM. https://github.com/UoS-SCCS/LWPM_Solving
22. Lee, P.J., Brickell, E.F.: An observation on the security of mceliece’s public-key cryptosystem. *Advances in Cryptology – EUROCRYPT ’88* **330**, 275–280 (1988)
23. Lenstra, A.K., Lenstra, H.W., Lovász, L.: Factoring polynomials with rational coefficients. *Mathematische Annalen* **261**(4), 515–534 (1982)
24. Leon, J.: A probabilistic algorithm for computing minimum weights of large error-correcting codes. *IEEE Transactions on Information Theory* **34**(5), 1354–1359 (1988)
25. Linderöth, J.T., Lodi, A., et al.: Milp software. *Wiley encyclopedia of operations research and management science* **5**, 3239–3248 (2010)
26. Löndahl, C.: Algorithms for Finding Low-Weight Polynomial Multiples. Ph.D. thesis, Lund University (2013)
27. Löndahl, C., Johansson, T.: Improved algorithms for finding low-weight polynomial multiples in $\mathbb{F}_2[x]$. *Designs, Codes and Cryptography* **73**, 625–640 (2014)
28. Lübke, O.: Igmaxhs—an incremental maxsat solver with support for xor clauses. arXiv preprint [arXiv:2410.15897](https://arxiv.org/abs/2410.15897) (2024)
29. May, A., Meurer, A., Thomae, E.: Decoding random linear codes in $\tilde{\mathcal{O}}(2^{0.054n})$. In: *Advances in Cryptology – ASIACRYPT 2011*. LNCS, vol. 7073, pp. 107–124. Springer (2011). https://doi.org/10.1007/978-3-642-25385-0_6
30. May, A., Ozerov, I.: On computing nearest neighbors with applications to decoding of linear codes. *EUROCRYPT 2015* (2015)
31. Meier, W.: Fast correlation attacks: Methods and countermeasures (2011)
32. Meier, W., Staffelbach, O.: Fast correlation attacks on certain stream ciphers. *Journal of Cryptology* **1**(3), 159–176 (1989)
33. Mouha, N., Wang, Q., Gu, D., Preneel, B.: Differential and linear cryptanalysis using mixed-integer linear programming. In: *International Conference on Information Security and Cryptology*. pp. 57–76. Springer (2011)
34. Prange, E.: The use of information sets in decoding cyclic codes. *IRE Transactions on Information Theory* **8**(5), 5–9 (1962)
35. Salom, M., Sendrier, N., Vasseur, V.: Sparse vector reconstruction from distance spectrum using soft information. In: *International Conference on Post-Quantum Cryptography*. pp. 99–129. Springer (2026)

36. Siegenthaler, T.: Cryptanalysts representation of nonlinearly filtered ml-sequences. In: Workshop on the Theory and Application of Cryptographic Techniques. pp. 103–110. Springer (1985)
37. Sinz, C.: Towards an optimal cnf encoding of boolean cardinality constraints. In: CP 2005 (2005)
38. Stern, J.: A method for finding codewords of small weight. Coding Theory and Applications pp. 106–113 (1989)
39. Țiplea, F.L., Lăzărescu, S.M.: On the low weight polynomial multiple problem. arXiv preprint arXiv:2410.10224 (2024)
40. Todo, Y., Isobe, T., Meier, W., Aoki, K., Zhang, B.: Fast correlation attack revisited. In: CRYPTO 2018 (2018)
41. Tseitin, G.S.: On the complexity of derivation in propositional calculus. In: Studies in Constructive Mathematics and Mathematical Logic (1968), reprinted/translations widely used; standard reference for Tseitin encodings
42. Vielma, J.P., Ahmed, S., Nemhauser, G.L.: Mixed integer linear programming formulations for probabilistic constraints. Operations Research Letters **40**(3), 153–158 (2012)

A A Representation-Based ISD Variant: BJMM

Stern’s algorithm is the canonical representative of classical ISD, but concrete ISD-based security estimates in the stream-cipher and code-based literature are nowadays derived from later representation-based algorithms, chiefly MMT [29] and BJMM [2]; BJMM is the usual reference point (e.g. in the syndrome-decoding estimator of Esser and Bellini [13] and in the analyses of the NIST code-based candidates). To rule out the objection that the gap we observe is an artefact of an older ISD variant, we also implement BJMM and run it under the same protocol as Stern (Section 5).

We prefer BJMM to the asymptotically faster nearest-neighbour or sieving variants because it fits the same information-set framework as Stern (so the reduction of Section 4 applies unchanged).

A.1 BJMM Description

BJMM keeps Stern’s outer structure: each iteration permutes the columns, brings the generator matrix into systematic form $G' = [I_k \mid A]$, selects a window B of ℓ redundancy coordinates, and searches for an information vector u of weight p with $(uA)|_B = 0$ and $\text{wt}(uG') \leq w$.

The difference is how u is built. Stern writes $u = u_1 + u_2$ with the u_i supported on disjoint halves of the information set, each of weight $p/2$, a unique decomposition merged by a collision on B . BJMM’s representation technique lets each u_i have support anywhere, with weight $p/2 + \varepsilon$, the ε shared positions cancelling in the sum ($\varepsilon = 0$ is MMT [29]). A target u then has $R = \binom{p}{p/2} \binom{k-p}{\varepsilon}$ representations, and since one suffices, about $\log_2 R$ extra linear constraints may be imposed on u_1 (namely $(u_1A)|_{B_2} = r$ on a sub-window $B_2 \subseteq B$, $|B_2| = \ell_2 \approx \log_2 R$), shrinking the intermediate lists by a factor of about 2^{ℓ_2} : this

is BJMM's asymptotic gain over Stern. Each list of vectors u_i is itself a Stern-type merge of two base lists, giving a depth-two search tree ending in a final merge on B and the weight test. The algorithm is given below, with parameters $(p, \varepsilon, |B|, \ell_2, L, T)$, where L and T mean the same as for Stern.

A.2 BJMM Codeword Search

Algorithm 6 BJMM [2] (depth 2, in our terminology)

Require: Generator matrix $G \in \mathbb{F}_2^{k \times n}$; weight target w ; parameters: p (even), ε (overlap), $|B|$ (window size), $\ell_2 \leq |B|$ (sub-window size), L (list cap), T (iterations)

Ensure: A non-zero codeword $c \in \mathcal{C}(G)$ with $\text{wt}(c) \leq w$, if found

- 1: $p_1 \leftarrow p/2 + \varepsilon$
- 2: **for** $t = 1$ **to** T **do**
- 3: Sample a random column permutation $\pi \in S_n$; compute a systematic form $G' = [I_k \mid A]$ of $G\pi$, recording π
- 4: Choose a window B of $|B|$ redundancy coordinates and a sub-window $B_2 \subseteq B$, $|B_2| = \ell_2$
- 5: Sample a random target $r \in \mathbb{F}_2^{\ell_2}$
- 6: **for** $i = 1, 2$ **do**
- 7: Split $\{1, \dots, k\}$ at random into two halves $I_{i,1}, I_{i,2}$
- 8: $\mathcal{L}_{i,1} \leftarrow \{x : \text{supp}(x) \subseteq I_{i,1}, \text{wt}(x) = \lceil p_1/2 \rceil\}$ (at most L elements)
- 9: $\mathcal{L}_{i,2} \leftarrow \{y : \text{supp}(y) \subseteq I_{i,2}, \text{wt}(y) = \lfloor p_1/2 \rfloor\}$ (at most L elements)
- 10: $\mathcal{L}_i \leftarrow \{x + y : x \in \mathcal{L}_{i,1}, y \in \mathcal{L}_{i,2}, ((x + y)A)|_{B_2} = r\}$ {merge on B_2 }
- 11: **end for**
- 12: **for all** $(u_1, u_2) \in \mathcal{L}_1 \times \mathcal{L}_2$ with $(u_1 A)|_B = (u_2 A)|_B$ **do**
- 13: $u \leftarrow u_1 + u_2$ {merge on B }
- 14: **if** $\text{wt}(u) = p$ **and** $1 \leq \text{wt}(uG') \leq w$ **then**
- 15: **return** $c \leftarrow (uG')\pi^{-1}$
- 16: **end if**
- 17: **end for**
- 18: **end for**
- 19: **return** fail

B Algorithms for Setting Up the Experiments

Algorithm 7: Random Polynomial over $\mathbb{F}_2$

Require: $n \in \mathbb{N}$, $w \in \{1, \dots, n+1\}$
Ensure: $P(X) = \sum_{i=0}^n a_i X^i$ with $w_H(P) = w$
1: $a_i \leftarrow 0$ for all $i = 0, \dots, n$
2: Choose randomly $S \subset \{0, \dots, n\}$ with $|S| = w$
3: **for** $i \in S$ **do**
4: $a_i \leftarrow 1$
5: **end for**
6: **return** $(a_0, \dots, a_n)$

Algorithm 8: Random Block Polynomial with Variable Length

Require: Degree $n \geq 0$, block length range $[L_{\min}, L_{\max}]$
Ensure: Polynomial $P(X)$ with contiguous block support
1: $a_i \leftarrow 0$ for all $i = 0, \dots, n$
2: $\ell \leftarrow \text{Uniform}(L_{\min}, L_{\max})$; $s \leftarrow \text{Uniform}(0, n - \ell + 1)$; $e \leftarrow s + \ell - 1$;
3: **for** $i = s$ to e **do**
4: $a_i \leftarrow 1$
5: **end for**
6: **return** $(a_0, \dots, a_n)$

C More on Experiments via MILP & MAXSAT-solving

C.1 Experimental results for random and non-random regimes

Table 5: Random regime, config. $(n, d, w) = (50, 10, 8)$: weight of $P(x)Q(x)$, CPU time (s), peak memory (MiB) and wall-time (s), over 250 instances.

No.	n	d	w	Solver	Weight			CPU (s)			Peak mem (MiB)			Wall (s)		
					Max	Avg	Med	Max	Avg	Med	Max	Avg	Med	Max	Avg	Med
1	50 ^v	10 ^f	8 ^f	MILP	8.0	8.0	8.0	0.5	0.02	0.02	0.13	0.09	0.09	0.59	0.03	0.02
2	50 ^v	10 ^f	8 ^f	IGMaxHS	8.0	8.0	8.0	0.31	0.11	0.1	2.08	0.18	0.17	0.3	0.11	0.1
3	50 ^v	10 ^f	8 ^f	PySat	8.0	8.0	8.0	0.03	0.01	0.0	65.78	65.61	65.63	0.44	0.42	0.42
4	50 ^v	10 ^f	8 ^f	Stern (223/250)	8.0	8.0	8.0	5.57	0.43	0.26	216.17	215.07	215.07	5.59	0.45	0.28
5	50 ^v	10 ^f	8 ^f	BJMM (230/250)	8.0	8.0	8.0	0.27	0.12	0.11	206.98	205.76	206.66	0.28	0.14	0.13
6	50 ^v	10 ^f	8 ^v	MILP	8.0	5.87	5.0	0.03	0.02	0.02	0.1	0.09	0.08	0.14	0.03	0.03
7	50 ^v	10 ^f	8 ^v	IGMaxHS	8.0	5.02	5.0	0.14	0.02	0.02	1.95	0.05	0.04	0.14	0.02	0.02
8	50 ^v	10 ^f	8 ^v	PySat	8.0	5.83	5.0	0.03	0.01	0.0	65.5	65.26	65.31	0.48	0.42	0.42
9	50 ^v	10 ^f	8 ^v	Stern (207/250)	8.0	4.55	5.0	13.73	0.37	0.2	221.62	221.5	221.51	14.25	0.75	0.59
10	50 ^v	10 ^f	8 ^v	BJMM (179/250)	8.0	4.01	5.0	0.83	0.25	0.24	213.08	208.26	212.47	0.84	0.27	0.26
11	50 ^f	10 ^f	8 ^v	MILP	8.0	5.34	5.0	0.09	0.02	0.02	0.11	0.09	0.09	0.13	0.03	0.03
12	50 ^f	10 ^f	8 ^v	IGMaxHS	8.0	5.1	5.0	0.14	0.02	0.02	1.95	0.05	0.04	0.15	0.02	0.02
13	50 ^f	10 ^f	8 ^v	PySat	8.0	5.86	5.0	0.03	0.0	0.0	65.6	65.4	65.48	0.71	0.43	0.42
14	50 ^f	10 ^f	8 ^v	Stern (104/250)	8.0	5.6	5.0	1.11	0.45	0.5	221.04	220.93	220.93	2.48	0.85	0.87
15	50 ^f	10 ^f	8 ^v	BJMM (71/250)	8.0	5.79	5.0	0.27	0.17	0.18	208.91	198.53	194.5	0.28	0.19	0.19

Note: For n, d, w , a superscript ^v marks a parameter that varies (up to the value shown) and ^f a parameter fixed to the value shown.

D Additional Experiments

This appendix we add additional experiments. All runs use the platform, instance sets, and solver configurations of Section 5 (see Appendix E for the

Table 6: Random regime, config. $(n, d, w) = (200, 60, 10)$: weight of $P(x)Q(x)$, CPU time (s), peak memory (MiB) and wall-time (s), over 250 instances.

No.	n	d	w	Solver	Weight			CPU (s)			Peak mem (MiB)			Wall (s)		
					Max	Avg	Med	Max	Avg	Med	Max	Avg	Med	Max	Avg	Med
1	200 ^v	60 ^f	10 ^f	MILP	10.0	10.0	10.0	0.1	0.05	0.05	0.17	0.13	0.13	0.2	0.06	0.06
2	200 ^v	60 ^f	10 ^f	IGMaxHS	10.0	10.0	10.0	14.55	2.55	1.96	5.04	2.71	2.7	14.9	2.55	1.96
3	200 ^v	60 ^f	10 ^f	PySat	10.0	10.0	10.0	1.89	0.11	0.05	75.66	70.58	70.9	12.36	0.54	0.46
4	200 ^v	60 ^f	10 ^f	Stern (250/250)	10.0	10.0	10.0	6.32	0.68	0.63	253.62	224.97	223.01	6.34	0.7	0.64
5	200 ^v	60 ^f	10 ^f	BJMM (250/250)	10.0	10.0	10.0	0.5	0.17	0.15	207.34	206.56	206.57	0.52	0.19	0.17
6	200 ^v	60 ^f	10 ^v	MILP	10.0	10.0	10.0	0.07	0.05	0.05	0.14	0.13	0.13	0.15	0.07	0.06
7	200 ^v	60 ^f	10 ^v	IGMaxHS	10.0	10.0	10.0	0.67	0.13	0.12	2.52	0.21	0.2	0.66	0.13	0.12
8	200 ^v	60 ^f	10 ^v	PySat	10.0	10.0	10.0	10.98	0.28	0.06	79.62	69.19	69.07	11.46	0.7	0.5
9	200 ^v	60 ^f	10 ^v	Stern (250/250)	10.0	10.0	10.0	17.72	0.68	0.55	271.75	230.14	227.47	18.27	1.06	0.93
10	200 ^v	60 ^f	10 ^v	BJMM (250/250)	10.0	10.0	10.0	0.5	0.18	0.16	208.92	207.82	207.82	1.04	0.21	0.18
11	200 ^f	60 ^f	10 ^v	MILP	10.0	10.0	10.0	0.07	0.05	0.05	0.14	0.14	0.14	0.18	0.08	0.07
12	200 ^f	60 ^f	10 ^v	IGMaxHS	10.0	10.0	10.0	0.36	0.13	0.12	2.52	0.21	0.2	0.36	0.13	0.13
13	200 ^f	60 ^f	10 ^v	PySat	10.0	10.0	10.0	5.17	0.16	0.06	77.88	68.98	68.93	5.61	0.6	0.49
14	200 ^f	60 ^f	10 ^v	Stern (250/250)	10.0	10.0	10.0	3.38	0.6	0.46	222.71	221.76	222.13	3.76	0.98	0.84
15	200 ^f	60 ^f	10 ^v	BJMM (250/250)	10.0	10.0	10.0	0.38	0.18	0.16	208.84	208.44	208.44	0.87	0.21	0.18

Table 7: Non-random (block) regime, config. $(n, d, w) = (250, 70, 10)$: weight of PQ , CPU time (s), peak memory (MiB) and wall-time (s), over 250 instances.

No.	n	d	w	Solver	Weight			CPU (s)			Peak mem (MiB)			Wall (s)		
					Max	Avg	Med	Max	Avg	Med	Max	Avg	Med	Max	Avg	Med
1	250 ^v	70 ^f	10 ^f	MILP	10.0	10.0	10.0	1.77	0.09	0.07	0.17	0.13	0.13	1.96	0.11	0.08
2	250 ^v	70 ^f	10 ^f	IGMaxHS	10.0	10.0	10.0	2.93	2.63	2.65	6.57	4.12	4.1	2.94	2.63	2.65
3	250 ^v	70 ^f	10 ^f	PySat	10.0	10.0	10.0	0.25	0.05	0.02	73.65	71.03	70.41	0.91	0.44	0.41
4	250 ^v	70 ^f	10 ^f	Stern (0/250)	N/A	N/A	N/A	TO	TO	TO	443.48	430.62	429.14	TO	TO	TO
5	250 ^v	70 ^f	10 ^f	BJMM (0/250)	N/A	N/A	N/A	33.18	17.32	16.5	33.2	17.34	16.52	197.7	195.89	195.99
6	250 ^v	70 ^f	10 ^v	MILP	10.0	7.61	8.0	0.08	0.04	0.04	0.15	0.12	0.13	0.15	0.06	0.06
7	250 ^v	70 ^f	10 ^v	IGMaxHS	2.0	2.0	2.0	0.44	0.17	0.17	2.7	0.25	0.24	0.45	0.17	0.17
8	250 ^v	70 ^f	10 ^v	PySat	8.0	2.1	2.0	0.23	0.06	0.02	72.47	70.47	70.67	0.84	0.45	0.4
9	250 ^v	70 ^f	10 ^v	Stern (250/250)	8.0	5.86	6.0	13.34	1.63	1.77	226.92	216.68	216.23	13.95	1.64	1.78
10	250 ^v	70 ^f	10 ^v	BJMM (243/250)	8.0	5.59	6.0	2.36	0.53	0.4	219.07	217.91	217.99	2.38	0.55	0.42
11	250 ^f	70 ^f	10 ^v	MILP	10.0	4.99	4.0	1.48	0.08	0.07	0.21	0.15	0.15	1.49	0.11	0.1
12	250 ^f	70 ^f	10 ^v	IGMaxHS	10.0	4.14	4.0	0.47	0.2	0.2	2.71	0.27	0.26	0.47	0.2	0.2
13	250 ^f	70 ^f	10 ^v	PySat	10.0	4.65	4.0	0.52	0.03	0.02	72.46	70.41	70.57	1.07	0.6	0.59
14	250 ^f	70 ^f	10 ^v	Stern (248/250)	10.0	4.87	4.0	117.28	20.4	10.96	223.7	220.05	220.18	117.29	20.42	10.97
15	250 ^f	70 ^f	10 ^v	BJMM (245/250)	10.0	4.91	4.0	5.0	1.08	0.6	226.71	226.05	226.33	5.02	1.11	0.62

thread settings), with a per-instance wall-clock limit of 120s for every solver, optimisation-based and ISD-based alike. A run that exceeds the limit is counted as unsolved and contributes no time or weight statistic; this is why we report, for each cell, the number of solved instances first.

D.1 Varying the target weight w at fixed (n, d)

Table 4 in the main text gives the solved counts and median times. Table 8 gives the full statistics (maximum, average, median of wall-time, CPU time and peak memory, over the solved instances) for the same runs: the 250 random instances of $(n, d) = (200, 60)$ of Section 5.6, with the weight bound decreased from $w = 10$

to $w = 6$. Since the instance set is the same for every w , an instance that is solved at weight w but not at weight $w - 1$ within the limit tells us directly where each solver stops being practical.

Table 8: Random regime, $(n, d) = (200, 60)$, 250 instances per w , 120s limit. “Solved” is the number of instances for which a multiple of weight $\leq w$ was returned within the limit; for $w = 10$ the statistics are over the runs returning a multiple, and for $w \leq 9$ over the runs ending in an infeasibility proof (MILP, IGMaHS, PySAT), respectively an inconclusive stop (Stern, BJMM).

w Solver	Solved	Wall (s)			CPU (s)			Peak mem (MiB)		
		Max	Avg	Med	Max	Avg	Med	Max	Avg	Med
10 MILP (CPLEX)	250	0.39	0.1	0.08	0.17	0.06	0.06	0.21	0.14	0.14
10 IGMaHS	250	0.36	0.13	0.13	0.35	0.13	0.13	2.52	0.21	0.2
10 PySAT	250	8.32	0.57	0.47	7.2	0.16	0.06	76.8	68.39	68.32
10 Stern	250	1.87	0.51	0.42	1.85	0.49	0.4	212.6	211.49	211.62
10 BJMM	250	0.37	0.18	0.17	0.35	0.16	0.15	206.24	205.88	205.88
9 MILP (CPLEX)	0	7.31	0.47	0.32	7.05	0.37	0.22	0.2	0.13	0.13
9 IGMaHS	0	0.5	0.12	0.1	0.5	0.11	0.1	2.45	0.13	0.12
9 PySAT	0	12.55	0.77	0.57	11.94	0.37	0.17	78.62	68.68	68.75
9 Stern	0	119.46	96.16	95.35	119.4	96.14	95.34	217.09	216.84	216.83
9 BJMM	0	3.16	3.03	3.02	3.15	3.02	3.01	192.56	192.49	192.49
8 MILP (CPLEX)	0	5.21	0.29	0.2	5.11	0.24	0.14	0.2	0.13	0.13
8 IGMaHS	0	0.33	0.1	0.09	0.31	0.09	0.09	2.44	0.12	0.11
8 PySAT	0	5.96	0.64	0.55	5.41	0.2	0.11	68.82	51.95	52.48
8 Stern	0	118.65	95.99	94.97	118.63	95.97	94.94	217.8	217.52	217.57
8 BJMM	0	3.9	3.04	3.02	3.89	3.03	3.0	192.85	192.78	192.79
7 MILP (CPLEX)	0	8.75	0.35	0.14	8.6	0.31	0.1	0.21	0.13	0.13
7 IGMaHS	0	0.3	0.08	0.08	0.3	0.08	0.08	2.43	0.11	0.1
7 PySAT	0	2.46	0.52	0.48	2.05	0.13	0.08	71.55	68.05	68.2
7 Stern	0	117.73	95.54	94.57	117.71	95.51	94.54	214.86	214.6	214.59
7 BJMM	0	3.31	3.07	3.04	3.3	3.05	3.03	196.47	193.59	193.07
6 MILP (CPLEX)	0	3.29	0.16	0.1	3.12	0.13	0.06	0.14	0.12	0.12
6 IGMaHS	0	0.29	0.07	0.06	0.29	0.07	0.06	2.42	0.1	0.09
6 PySAT	0	1.69	0.51	0.49	0.69	0.07	0.06	68.07	65.61	66.84
6 Stern	0	119.69	96.79	95.54	119.66	96.75	95.53	215.47	215.3	215.33
6 BJMM	0	5.67	3.03	3.01	5.52	3.02	2.99	193.23	193.19	193.18

D.2 ISD at further parameter points

Table 9 reports both ISD baselines, Stern (Section 4.2) and BJMM (Appendix A), at the remaining parameter points used for SAT solvers, in both the random and the block regime, together with the optimisation solvers on the same instances. The BJMM parameters were set to $(p, \varepsilon, |B|, \ell_2) = (4, 1, 10, \text{None})$, the smallest values for which the representation count R exceeds one; for these bounded-degree, low-weight instances R is small and BJMM is not expected to be dra-

matically faster than Stern, which is what we observe. We present the results only for the instances that did not exceed the imposed time limit.

Table 9: Solved instances out of 250 (median wall-time in seconds over solved instances), 120 s limit, all solvers, random and block regimes.

Regime (n, d, w)	MILP (CPLEX)	IGMaxHS	PySAT	Stern	BJMM
random ($100^v, 20^f, 8^f$)	250 (0.03)	250 (0.36)	250 (0.42)	250 (0.21)	242 (0.14)
random ($100^v, 20^f, 8^v$)	250 (0.03)	250 (0.03)	250 (0.43)	230 (0.65)	216 (0.16)
random ($100^f, 20^f, 8^v$)	250 (0.04)	250 (0.03)	250 (0.43)	154 (2.45)	125 (0.52)
random ($250^v, 70^f, 10^f$)	250 (0.07)	250 (0.13)	250 (0.49)	250 (0.9)	250 (0.21)
random ($250^v, 70^f, 10^v$)	250 (0.08)	250 (0.15)	250 (0.51)	250 (1.32)	250 (0.2)
random ($250^f, 70^f, 10^v$)	250 (0.09)	250 (0.15)	250 (0.47)	250 (1.26)	250 (0.2)
random ($300^v, 80^f, 10^f$)	250 (0.13)	250 (4.11)	250 (0.5)	250 (1.34)	250 (0.24)
random ($300^v, 80^f, 10^v$)	250 (0.12)	250 (0.2)	250 (0.49)	250 (1.8)	250 (0.22)
random ($300^f, 80^f, 10^v$)	250 (0.15)	250 (0.2)	250 (0.52)	250 (1.71)	250 (0.23)
random ($350^v, 90^f, 10^f$)	250 (0.17)	250 (4.51)	250 (0.57)	250 (1.89)	250 (0.35)
random ($350^v, 90^f, 10^v$)	250 (0.18)	250 (0.23)	250 (0.52)	250 (2.18)	250 (0.26)
random ($350^f, 90^f, 10^v$)	250 (0.21)	250 (0.23)	250 (0.53)	250 (2.13)	250 (0.4)
block ($100^v, 20^f, 8^f$)	250 (0.03)	250 (0.34)	250 (0.39)	234 (0.4)	234 (0.2)
block ($100^v, 20^f, 8^v$)	250 (0.03)	250 (0.04)	250 (0.41)	250 (0.22)	218 (0.16)
block ($100^f, 20^f, 8^v$)	250 (0.03)	250 (0.04)	250 (0.58)	250 (0.49)	218 (0.37)
block ($150^v, 40^f, 10^f$)	250 (0.04)	250 (0.88)	250 (0.39)	0 (–)	0 (–)
block ($150^v, 40^f, 10^v$)	250 (0.03)	250 (0.09)	250 (0.4)	250 (0.54)	240 (0.23)
block ($150^f, 40^f, 10^v$)	250 (0.05)	250 (0.1)	250 (0.58)	250 (1.87)	247 (0.43)

D.3 Real feedback polynomials

The random and block families of Section 5.1 are controlled families: random sparse polynomials model the generic case assumed in most analyses, and block polynomials probe the effect of structure, since attacks in the literature were mounted on feedback polynomials that were far from random. The instances that matter for security claims, however, are the feedback polynomials of actual LFSR-based designs. We therefore ran all solvers on the following polynomials.

- **Didier–Laigle–Chapuy (2007).** The degree-53 polynomial used as the worked example in [10], $P = x^{53} + x^{47} + x^{45} + x^{44} + x^{42} + x^{40} + x^{39} + x^{38} + x^{36} + x^{33} + x^{32} + x^{31} + x^{30} + x^{28} + x^{27} + x^{26} + x^{25} + x^{21} + x^{20} + x^{17} + x^{16} + x^{15} + x^{13} + x^{11} + x^{10} + x^7 + x^6 + x^3 + x^2 + x + 1$, for which that paper reports low-weight multiples found by a dedicated discrete-logarithm method; this gives a reference point for the weights our solvers reach.
- **GSM A5/1.** The three register polynomials $x^{19} + x^{18} + x^{17} + x^{14} + 1$, $x^{22} + x^{21} + 1$ and $x^{23} + x^{22} + x^{21} + x^8 + 1$ [7], and their product (degree 64), which is the polynomial relevant to correlation attacks on the combined output.

- **Bluetooth E0.** The four register polynomials of degrees 25, 31, 33 and 39 [8], $x^{25} + x^{20} + x^{12} + x^8 + 1$, $x^{31} + x^{24} + x^{16} + x^{12} + 1$, $x^{33} + x^{28} + x^{24} + x^4 + 1$, $x^{39} + x^{36} + x^{28} + x^4 + 1$, and their product (degree 128).

Because these polynomials have low degree, the multiplier degree d can be pushed much further than in the random experiments: we use $d \in \{100, 300, 500\}$. Here, every solver is run in an anytime fashion under the same 120 s budget. In the case of the MILP approach, several weight bounds were tested, progressively decreasing w in order to obtain a solution. Since no lower-weight solution was found within the time limit, the best weight successfully attained by MILP was retained. The ISD solvers were similarly evaluated over decreasing target weights. For each solver, Table 10 reports the lowest weight that the solvers reached within the time limit and iteration budget, denoted w^* (a per-solver value, not a shared target), together with the wall-time at which that weight was first attained. No optimality certificates are claimed in this table: for every solver, w^* is only the best weight found within the allocated search budget. A useful sanity bound is $\text{wt}(P)$ itself, attained by the trivial multiplier $Q = x^d$; e.g. $\text{wt}(P) = 31$ for the Didier–Laigle–Chapuy polynomial, 5 for E0 R_4 , and 27 and 49 for the A5/1 and E0 products. The optimisation algorithms here run in the same bounded-weight (feasibility) mode as in the rest of the paper: the MILP driver has a constant objective, a solutions limit of one (`mip.limits.solutions=1`) and the bounds $\lceil (w-1)/2 \rceil \leq \text{wt}(PQ) \leq w$, and the SAT drivers return the first model under the hard bound. Consequently, w^* is the weight of the first solution returned under the bounds swept, and can exceed the trivial bound $\text{wt}(P)$ (as for E0 R_4) or coincide with it (as on the two product polynomials). Thus, the table should be interpreted as reporting the lowest weight reached within the search budget.

Table 10: Real feedback polynomials. For each solver, w^* is the lowest weight of a multiple that this solver found within a 120 s budget (the target is not shared across solvers), and the accompanying column gives the wall-time in seconds at which that weight was first attained.

Polynomial	deg P	d	w^*	MILP	w^*	IGMaxHS	w^*	PySAT	w^*	Stern	w^*	BJMM
Didier–LC	53	100	23	1.76	17	1.72	22	0.69	17	0.84	12	0.52
Didier–LC	53	300	25	0.89	25	1.10	24	0.41	19	4.04	12	0.49
A5/1 R_1	19	100	5	0.10	5	0.22	5	0.39	5	32.24	4	0.26
A5/1 product	64	300	27	0.08	27	0.99	27	1.28	20	5.96	15	2.61
E0 R_4	39	100	8	0.07	5	0.21	8	0.36	8	0.80	5	0.34
E0 product	128	500	49	0.23	49	2.64	48	2.69	47	15.42	38	11.37

E Solver Configuration and Implementation Effort

Configurations used. CPLEX (via the Sage `MixedIntegerLinearProgram` interface) was run with default MIP emphasis (balanced), default heuristic frequency and default cut generation, the default/automatic number of threads (as in Section 5, which can multi-threaded but we do not control this and let

is be chosen by the default parameters), and a time limit of 120 s ; the MILP driver runs in feasibility mode: a constant objective, a solutions limit of one (`mip.limits.solutions=1`), the hard bound $\text{wt}(PQ) \leq w$, and a lower bound $\text{wt}(PQ) \geq \lceil (w-1)/2 \rceil$ that excludes near-trivial products (its infeasible answers therefore certify non-existence in the range $\lceil (w-1)/2 \rceil, w$]; the SAT-based UNSAT answers certify the full range $[1, w]$); PySAT used CryptoMiniSat 5 with native XOR clauses and a CNF cardinality encoding for the Hamming-weight constraint (hard $\text{Equals}(w)$, respectively $\text{AtMost}(w)$ plus $\text{AtLeast}(1)$), all defaults; IGMMaxHS [28] was run as distributed, with hard clauses only (sequential-counter cardinality encoding), and it exposes no user-tunable parameters beyond the time limit. Our Stern and BJMM implementations are in Python and were run with the parameters given in Appendix A. The BJMM parameters were set to $(p, \varepsilon, |B|, \ell_2) = (4, 1, 10, \text{None})$, while the Stern parameters were set to $(p, |B|, N_{\text{iter}}, L_{\text{max}}) = (2, 20, 200, 5000)$.

Why defaults. The comparison is deliberately between unoptimised uses of each paradigm. The point of the paper is not that a tuned CPLEX beats a tuned ISD, but that a cryptanalyst with no solver expertise, feeding the encodings of Section 3 to an off-the-shelf solver, obtains results that compete with a dedicated LWPM pipeline. Tuning the optimisation side would only widen the gap.

A small tuning experiment. To quantify how much is left on the table, we re-ran the $(200, 60)$, $w \in \{8, 7, 6\}$ instances of Appendix D.1 with CPLEX under three non-default settings from [18]: MIP emphasis set to feasibility (`MIPEmphasis=1`), heuristic effort increased (`HeurFreq=10`), and aggressive cut generation (in particular, zero-half cuts were considered due to the binary and parity-related structure; cover cuts are necessary due to the cardinality constraint that imposes Hamming weight; Gomory fractional and mixed-integer rounding (MIR) cuts were also considered as mechanisms for improving LP relaxation). Table 11 reports the solved counts and median times.

Table 11: CPLEX under non-default settings on the random $(200, 60)$ instances: solved out of 250 (median wall-time, s).

Setting	Solved	$w = 10$	Solved	$w = 9$	Solved	$w = 8$	Solved	$w = 7$	Solved	$w = 6$
default	250	0.08	0	0.32	0	0.2	0	0.14	0	0.1
<code>MIPEmphasis=1</code>	250	0.07	0	0.24	0	0.42	0	1.2	0	0.9
heuristics up	250	0.27	0	0.64	0	1.26	0	1.0	0	0.71
cuts aggressive	250	0.07	0	0.55	0	0.55	0	0.49	0	0.4

Implementation effort and fairness. Our ISD code is research-grade, single-threaded Python; CPLEX and CryptoMiniSat are heavily engineered tools, and CPLEX may use several threads (Section 5). Both asymmetries favour CPLEX; we bound their effect. First, comparisons are on wall-clock time, and the single-threaded MAXSAT solvers show the same qualitative behaviour as CPLEX, so the picture does not rest on CPLEX’s parallelism. Second, where ISD fails it fails structurally: in Table 4 the ISD baselines stop without an answer at budget exhaustion, an outcome a faster implementation would reach sooner but not change, since ISD has no certificate of non-existence at any speed. Third,

ours is the comparison a practitioner faces: a pipeline that must be implemented against a tool that need not be. The implementation gap remains a limitation, stated in Section 6.

F On Hybrid Optimisation–Decoding Approaches

We do not evaluate hybrids of the two paradigms: the point of this work is to isolate each on its own, and knowing which component to trust per regime is the prerequisite for a sensible hybrid. Our results suggest three, left to future work.

- **Seeding ISD with optimisation solutions.** The support of a multiple found quickly by MILP/MAXSAT under a loose bound can bias the ISD column permutation (keeping that support outside the information set), turning Stern or BJMM into a local improvement step.
- **ISD-derived bounds for branch-and-bound.** A weight from a fast ISD run at large w is a valid upper bound on the MILP objective and can be passed to CPLEX as an incumbent, pruning the search tree.
- **Optimisation as the collision step.** BJMM’s final merge on B is a small structured constraint problem; a SAT solver in place of hashing would trade memory for time when the lists are large.